RBCTest: Leveraging LLMs to Mine and Verify Oracles of API Response Bodies for RESTful API Testing

Hieu Huynh*
Katalon Inc.
Ho Chi Minh city, Vietnam
minhhieu2214@gmail.com

Quoc-Tri Le*
Katalon Inc.
Ho Chi Minh city, Vietnam
lqtri691@gmail.com

Tu Nguyen*
University of Science
Vietnam National Univ.
Ho Chi Minh city, Vietnam
23C15017@student.hcmus.edu.vn

Viet Nguyen
University of Science
Vietnam National Univ.
Ho Chi Minh city, Vietnam
21C11045@student.hcmus.edu.vn

Vu Nguyen†
University of Science
Vietnam National Univ.
Ho Chi Minh city, Vietnam
nvu@fit.hcmus.edu.vn

Tien N. Nguyen
Computer Science Department
University of Texas at Dallas
Dallas, Texas, USA
tien.n.nguyen@utdallas.edu

Abstract

In API testing, deriving logical constraints on API response bodies to be used as oracles is crucial in generating test cases and performing automated testing of RESTful APIs. However, existing approaches are restricted to dynamic analysis in which oracles are extracted via the execution of APIs as part of the system under test. In this paper, we propose a complementary LLM-based, static approach in which the constraints for API response bodies are mined from API specifications. We leverage large language models (LLMs) to comprehend the API specifications, mine constraints for response bodies, and generate test cases. To reduce LLMs' hallucination, we apply an Observation-Confirmation (OC) scheme which uses initial prompts to contextualize constraints, allowing subsequent prompts to more accurately confirm their presence. Our empirical results show that RBCTest with OC prompting achieves high precision in constraint mining with the average from 85.1%–93.6%. It also performs well in generating test cases from mined constraints, with a precision from 86.4%–91.7%. We also use the test cases generated by RBCTest to detect 46 mismatches between the API specification and actual response data for 19 real-world APIs. Four of the mismatches were, in fact, reported in developers' forums.

CCS Concepts

• **Software and its engineering**;

Keywords

AI4SE, Large Language Models, REST APIs, Automated API Testing

ACM Reference Format:
Hieu Huynh, Quoc-Tri Le, Tu Nguyen, Viet Nguyen, Vu Nguyen, and Tien N. Nguyen. 2026. RBCTest: Leveraging LLMs to Mine and Verify Oracles of API Response Bodies for RESTful API Testing. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26), April 12–18, 2026, Rio de Janeiro, Brazil*. ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3744916.3787775

*Co-first authors, equal contributions
†Corresponding author.

ICSE '26, Rio de Janeiro, Brazil

ACM ISBN 979-8-4007-2025-3/2026/04
https://doi.org/10.1145/3744916.3787775

1 Introduction

By adhering to the principles of Representational State Transfer (REST), the RESTful APIs provide a standardized way for interoperability among components and software systems. RESTful API testing helps identify and resolve several issues, ensuring that APIs perform as expected [12, 14, 18, 30, 33, 39]. Among techniques for API testing, black-box testing uses the OpenAPI Specification (OAS) as a basis to generate test cases and data [23, 28, 33]. The state-of-the-art API testing approaches are focused on status code [5, 6, 28, 39], schema validation [5, 6, 39], and rule extraction from OAS [24]. Existing static code validation approaches define an oracle for a test case by validating whether the response status code matches the expected value. In contrast, *schema validation* ensures the correctness of the response data by checking it against a specified schema.

While status code and schema validation effectively cover aspects of data representation and status checking, they may overlook the *logical correctness and validity of the response data from the APIs*. For example, an API request for a customer older than 18 receiving a response for one younger than 18 would not be detected by just validating the status code or schema. The state-of-the-art approaches for mining constraints on API response bodies focus only on *dynamic analysis* as the constraints are extracted from the execution data of the system under test (SUT). AGORA/AGORA+[10, 13] extends Daikon [20], a dynamic instrumenter, to detect invariants during the execution of the APIs within the SUTs.

This paper presents RBCTest, a novel LLM-based approach to mine the *constraints of API response bodies* from the API specification. To mine constraints, we leverage the ability of large language models (LLMs) to comprehend textual descriptions in API specifications. Constraints on API response bodies are inferred from different sources, including response properties, response schema, operations, and request parameters. We also harness LLMs' proficiency to create test cases from the mined constraints to verify if the SUT correctly returns the content satisfying the mined constraints. Moreover, we apply an *Observation-Confirmation* scheme by dividing

the task of mining constraints into two phases: observation and confirmation. The initial prompt contextualizes the description of constraints, enabling the next prompt to more accurately decide their presence. Moreover, LLMs might have hallucinations. Thus, we enhance RBCTest with two extra mechanisms. First, before requesting the LLM to make observations concerning constraints on parameters, we perform *a filtering process* to keep only the valid ones. Second, after generating test cases for mined constraints, we add a *semantic verifier* to verify those test cases against the examples specified in the OAS file. The idea is that such examples tend to be correct because they illustrate the descriptions on the data. For example, the OAS could give "March" as a valid month. If such a correct example does not pass a test case generated by RBCTest based on the mined constraint(s), the test case must be incorrect, which is caused by incorrect constraint(s). Thus, our verifier will discard them, leading to improved precision.

We evaluate RBCTest on two datasets, AGORA/AGORA+ [13] and our own dataset collected from 8 real-world API services consisting of 65 endpoints and 90 operations [7]. Our empirical results show that RBCTest achieves high precision in constraint mining with 93.6% on average for the RBCTest dataset and 85.1% for the AGORA/AGORA+ dataset. Precision in test generation for constraint validation is from 86.4%–91.7%. We also use its generated tests to automatically verify mined constraints and detect mismatches between the API specifications and actual execution of the SUTs. A detected mismatch indicates a fault in a SUT or that the SUT does not match its specification. We report 46 actual faults found in 19 real-world applications, including 4 issues reported by users on GitLab Forum [1–4]. This paper makes the following contributions:

1. RBCTest: An LLM-based method to extract constraints from API specifications and to generate tests to validate API responses.

2. A manually-verified benchmark with ground-truth constraints for API response bodies of 8 real-world API services is available [7] for future research on API testing approaches.

3. An extensive evaluation showing RBCTest's capabilities in constraint or oracle mining and test generation for API testing.

2 Motivation

Let us consider an example with Stripe, an online payment service. Figure 1(b) partially shows a description from the API specification (OAS), detailing the GET operation for retrieving charge records within a time interval (lines 9-10), which is defined by the `gt` and `lt` parameters (i.e., the lower and upper bounds). Users can also specify the customer for whom they wish to retrieve charging history (lines 15-17). A successful request returns a response with a status code of `200` and a response body with data. The structure of the response data (in Figure 1(a)) consists of a list of `charge` objects, each associated with various properties, e.g., amount, created timestamp, currency, and others. For instance, a GET request to the `/v1/charges` endpoint with parameters like `'created[gt]=1679090500&customer=cus_idA'` returns a list of charges that satisfy the given conditions. An example response object is in Figure 2. The response's content is constrained by the actual input parameters in the request. Thus, a complete testing process needs to verify the response content and the returned status code. For example, a test case can check if the `created` field of a returned charge object falls within the interval and ensuring that the `customer` field matches the requested ID.

```
1  (a) charge:
2      description : The 'charge' object represents an attempt to move
           money into your account.
3      properties :
4          amount:
5              description : A positive integer can be up to eight digits .
6              type: integer
7              example: 99999999
8          created :
9              description : Time at which the object was created. Measured in
                   seconds since the Unix epoch.
10             type: integer
11         currency:
12             description : Three lowercase letters .
13             type: string
14             example: usd
15         customer:
16             description : ID of the customer this charge is for if existed .
17             type: string ...
```

```
1  (b) paths:
2    /v1/charges:
3      get:
4        description : Returns a list of charges you have created. The
             charges are returned in sorted order ...
5        parameters:
6            name: created
7            description : Only return charges that were created during the
                 given date interval .
8            schema:
9              anyOf:
10               - properties :
11                   gt (integer)
12                   lt (integer)
13           name: customer
14           description : Only return charges for the customer specified by
                 this customer ID.
15           schema:
16             type: string ...
```

Figure 1: (a) Schema of a response for 'charge' API in the project Stripe described in a OAS file and (b) a simplified Description for the GET charges API operation from Stripe.

```
1 {
2   "id": "ch...15",
3   "object": "charge",
4   "customer": "cus_id",
5   "amount": 1099,
6   "created": 1679090539,
7   "currency": "usd",...
8 }
```

Figure 2: A Response's Body from a GET Request for Stripe.

Observation 1 (Constraints from input parameters). *The response data is constrained by the input parameters from the request. When testing, in addition to verifying the status code, testers need to verify the response bodies returned from the APIs.*

In Figure 1(a), the descriptions on the properties of the returned `charge` objects define certain constraints on the attributes. For example, the `amount` property must be positive, with a max value of eight digits. The `currency` attribute has a three-letter lowercase code.

Observation 2 (Constraints within response body). *Natural language descriptions on operations express logical constraints on operations, their responses, formatting requirements, or value range limitations that must be validated during API testing.*

The OAS file often includes examples that illustrate specific constraints on data or data types. For instance, in Figure 1(a), the file provides an example of $999,999.99 as a positive number for the property `amount`. A mined constraint that does not match its corresponding example may be incorrect. To validate that, we utilize the generated test case for the constraint.

Observation 3 (Verification with examples). *The illustrating examples in the description can be used to verify against the generated test cases, i.e., the validity of the mined constraints.*

3 Related Work

Recent surveys on API testing [19, 21, 27, 31, 32, 34, 38] reveal a trend towards automation adoption. AI/ML are used to enhance various aspects of API testing including of generating test cases [18, 33, 39], realistic test inputs [12], and identify defects early in the development [15–17, 30, 36, 43–46]. In API test case generation, validation typically falls into two main categories.

Status Code Validation: Each HTTP request is returned with a response containing a status code and data. The status code, a 3-digit integer, indicates the outcome of the HTTP request. `2xx` codes signify a successful request. Conversely, `4xx` codes indicate errors, e.g., a bad syntax request or invalid input values. For instance, in testing the API `'GET/user_information'` with various valid and invalid user IDs, testers would expect the API to return status codes `200` or `404`. This validation method is used in automated tools, e.g., Postman [6], Katalon [5], RestTestGen [39], and KAT [28].

Schema Validation: This verifies the presence of all required properties and the consistency of property data types with specifications. Tools, e.g., RestTestGen [39], leverage external libraries, such as 'swagger-schema-validator', to facilitate schema validation.

While status code and schema validation cover data representation and status checking, they *overlook the logical correctness and validity in the API responses*. For instance, if an API request for a charge in 2025 returns one from 2024, or if the amount is negative, these would not be detected by validating the status code or schema.

AGORA/AGORA+ [10, 13] extends Daikon [20] to infer the invariants from the execution data of the SUT using APIs. SATORI [11] uses LLMs to infer API's expected behavior by analyzing the properties of the response fields of its operations. It does not have Observation-Confirmation (OC) scheme as in RBCTest.

KAT [28] fully automates API testing using GPT and an input OpenAPI specification, building dependency graphs and generating test scripts. Kim *et al.* [26] applied GPT to enrich specifications with rule explanations and example inputs. Before LLMs, ARTE [12] used NLP to generate API test inputs. Morest [30] proposed model-based RESTful API testing with a dynamic Property Graph, improving code coverage and bug detection. RESTler [18] introduced stateful fuzzing of REST APIs, inferring dependencies and generating tests guided by service responses. LlamaRestTest [25] employs fine-tuned and quantized Llama3-8B model using mined datasets of REST API example values and inter-parameter dependencies, to generate realistic test inputs and uncover inter-parameter dependencies.

4 Key Ideas

We design RBCTest, an LLM-based, static approach for mining the constraints of API response bodies and then generating test cases.

Key Idea 1 [Leveraging LLMs to Mine Constraints in APIs' response bodies]. RBCTest is a novel LLM-based static approach to mine the constraints for API's response bodies from the OAS. The constraints on the response data can be found in the specification in either the descriptions of the *operations* (e.g., line 5 of Figure 1(a)) or the descriptions of the *schema for the response data* (lines 2–24 in Figure 1(b)). For example, the description in Figure 1 states that *'the charges are returned in a sorted order with the most recent ones appearing first'*. The schema for the returned values in Figure 1 also provides us several constraints on the *parameters* (lines 5–6) as well as the description of the returned object (line 2). For example, the amount of a charge is a positive number with up to 8 digits and such value must be of integer. In brief, RBCTest is useful in scenarios where APIs and their execution are unavailable or unreliable. For example, this occurs when an API specification is available, but the APIs and their underlying systems are still under development.

Key Idea 2 [Observation-Confirmation scheme on LLMs for constraints discovery and test generation]. For the task of extracting constraints from API specifications, we utilize the ability of LLMs to comprehend natural language descriptions found in API specifications for constraint mining on the APIs and their parameters. Our experiment showed that direct use of LLMs for constraint mining yields sub-optimal performance. To improve it, we apply an Observation-Confirmation scheme in which the initial result returned from the LLMs will be fed back to themselves in a confirmation prompt to provide better contexts on the constraints.

Key Idea 3 [Generating test cases for the constraints on response bodies]. Our goal is to advance beyond current API testing techniques: we also generate test cases to evaluate the mined constraints. For instance, a test case is generated to verify that the list of charges returned by the API endpoint 'v1/charges' is sorted in reverse chronological order. Another example includes generating a test case to validate the format/value of the returned charge amount.

Key Idea 4 [Filter and Semantic Verifier]. Before asking the LLM to analyze constraints on parameters, we perform filtering to remove invalid constraints. After generating test cases based on the mined constraints, we introduce a semantic verifier to check these test cases against the examples in the OAS. The rationale is that these examples are typically accurate in the specification. For instance, an example of $999,999.99 is used to illustrate a positive number for `amount`. If a test case fails to validate a given correct example, it suggests that the mined constraint may be incorrect.

5 Mining Constraints for API Response Bodies

The specification for an API endpoint includes two main parts: the *request specification* (the **input** of the API) and *response schema specification* (the **output** of the API). (1) A request specification determines how to call an API endpoint, detailing the required *input parameters*, their roles, and the parameters within the request body along with their respective roles. (2) The response schema specification provides instructions on the *response body*, including each property in the response data, its description, datatype, nullability, and other details. These parts are the target for constraint mining.

5.1 Constraints from Request Specifications

Figure 3 describes the process of mining constraints. Constraints for response bodies may exist in descriptions of operations, parameters, and other information defined in the request specifications.

5.1.1 Request-Response Constraint Mapping. The response data may contain a property that reflects a constraint available in request parameters. Thus, we *identify pairs consisting of a request parameter that includes a constraint and a response data property that reflects*

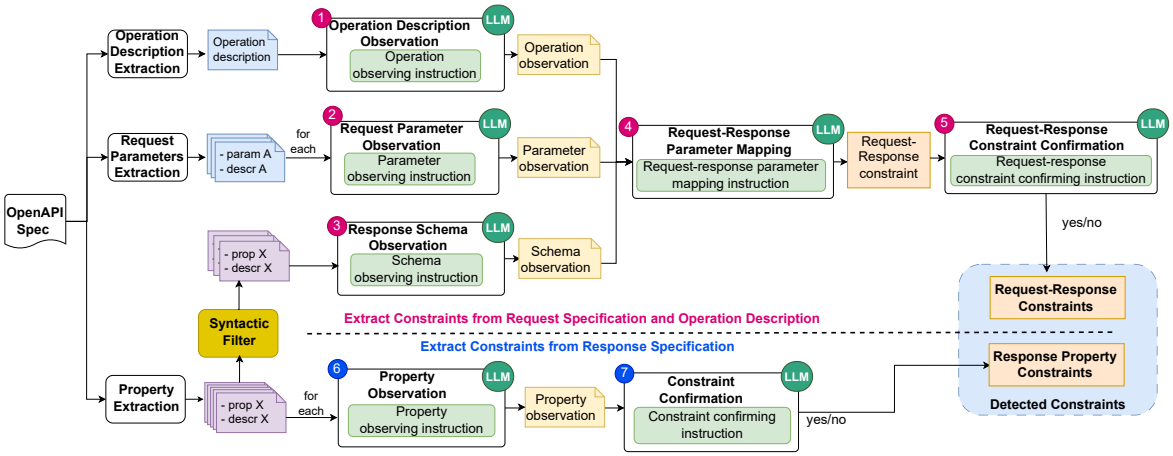

Figure 3: Mining Constraints in APIs' Response Bodies with LLMs

```
PARAMETER_SCHEMA_MAPPING_PROMPT = '''Given a request
      parameter and an API response schema, check if there is
       a matching property in the API response schema.
Request parameter for {method} - {endpoint}:
{parameter}: {description}

Follow these steps to find the matching property: {
      Instructions for chain-of-thought steps}

Schema specification {schema}: {schema_observation}
Confirm if the request parameter has a matching property in
      the response schema: ...
Identify the corresponding property name of the provided
      request parameter in the schema. ...
If a matching property exists, explain it in this format:...
```

Figure 4: Request-Response Parameter Mapping Observation

```
MAPPING_CONFIRMATION = '''{System prompt}
The request parameter's information:
- Operation: {method}
- Parameter: {parameter_name}
- Description: {description}
The corresponding property's information:
- Resource: {schema}
- Corresponding property: {corresponding_property}
{Instructions for chain-of-thought steps}
Answer format: ...'''
```

Figure 5: Request-Response Constraint Confirmation

this constraint (Figure 4). For instance, to verify a "customerID" constraint, the response data should contain a property that identifies the customer. Thus, the required pair is <"customerID", "customer">. If the response data does not have a field to represent a constraint, that constraint is disregarded as it cannot be validated.

5.1.2 Detailed Process. The process of extracting constraints from request parameters encompasses four steps: (1) description extraction, (2) **observation** (1-3 from Figure 3), (3) Request-Response constraint mapping (4), and (4) constraint **confirmation** (5).

Algorithm 1 provides the details. For each API request, a prompt is made to gather observations on the response schema and operations associated with this request (lines 3–4, Algorithm 1). With each request parameter (line 5), we engage the LLM to offer observations on the parameter using its description (line 12, Algorithm 1). These parameter observations support the next step:

Algorithm 1 Extract Constraints from Request Specifications

Input: API_spec(dict): obj of entire API specification.
req_spec(dict): specification obj of a certain request.
resp_schema(dict): schema obj of the associated response.
Output: list of request-response constraint properties

```
1:  function getConstrFromReqParas(API_spec, req_spec, resp_schema)
2:    reqRespConstraints ← []
3:    respSchemaObser ← LLM().respSchemaObser(resp_schema)
4:    operationObservation ← LLM().operationObser(desc)
5:    for each param in req_spec do
6:      desc ← req_spec[param]["desc"]
7:      if req_spec[param]["desc"] = NULL then
8:        desc ← findExactMatchParameter(API_spec, param)
9:        if desc = NULL then
10:         continue # Skip this param
11:     # Use LLM to find constraint for this param
12:     paramObser ← LLM().parameterObser(param, desc)
13:     answer, corrProp, explain ← LLM().reqRespMapping
          (param, desc, paramObser, respSchemaObser)
14:     if answer = TRUE then
15:       confirmation ← LLM().confirmReqRespMapping
    (param, corrProp, explain)
16:       if confirmation = TRUE then
17:         reqRespConstraints.append((param, corrProp))
18:   return reqRespConstraints
19: end function
```

Request-Response parameter mapping (Block 4, line 13). We guide the LLM through a two-step reasoning process using a tailored prompt (Figure 4). The LLM receives a brief description of the request parameter, including its details and observations (2), ensuring an understanding of its intent and constraints. We then present the response schema, observations from Block 3, and ask the LLM to identify a matching property. A match occurs when the request parameter filters response data or both share the same meaning.

From this two-step reasoning, we require the LLM to answer three questions: (1) Is there a matching property in the response schema? (2) What is this property? (3) How does the request parameter influence this property?

Algorithm 2 Extract Constraints from Response Specifications

Input: API_spec(dict): obj of entire API specification.
response_schema(dict): schema obj of a certain response.
knowledge_base(dict): gained knowledge.
Output: list of constraint properties.

```
1: function getConstraintInsideResponseSchema(API_spec, response_
   schema, knowledge_base)
2:     constraint_properties ← []
3:     for each prop in resp_schema do
4:         desc ← resp_schema[prop]["description"]
5:       +resp_schema[prop]["type"]+resp_schema[prop]["format"]
6:       +resp_schema[prop]["example"]
7:         if desc = NULL then
8:             desc ← exactMatchProp(API_spec, prop)
9:             if desc = NULL then
10:                continue # Skip this property
11:        if prop in knowledge_base then
12:            if knowledge_base[prop] = TRUE then
13:                constraint_properties.append(prop)
14:        else
15:            datatype ← response_schema[prop]["datatype"]
16:            prop_obser ← LLM().propertyObser(prop, datatype, desc)
17:            constraint_confirmation ← LLM().constraint_confirmation
   (prop, datatype, desc, prop_obser)
18:            if constraint_confirmation = TRUE then
19:                constraint_properties.append(prop)
20:            # Add this property to knowledge base
21:            knowledge_base[prop] ← constraint_confirmation
22:    return constraint_properties
23: end function
```

If the answer to (1) is false, i.e., no property reflects this request parameter, we disregard this parameter. Otherwise, we proceed to the final prompt (Figure 5), which is the confirmation of the mapping (lines 14–17, Block 5). We present the pair of the request parameter and the matched property from Block 4, and ask the LLM to confirm the accuracy of this mapping (line 15). This step aims to minimize the occurrence of false positives. Finally, the validated pairs of request parameters and response properties are stored as Request-Response constraints (lines 16–17).

5.2 Constraints from Response Specifications

We examine descriptions, data formats, and examples in the response schema specification, as they may provide constraints for a property. Each endpoint includes a response schema that guides clients on parsing returned data, structuring the response object with properties, descriptions, data formats, and examples (Figure 2).

Algorithm 2 shows our constraint-mining algorithm for response specifications. It first extracts descriptions, data formats, and examples for a property (lines 4-10). If such information exists, we check the knowledge base of LLM-identified constraints to avoid redundant queries (lines 11-13). If found, we reuse the stored data; otherwise, we prompt the LLM to extract constraints (lines 15-21).

Observation-Confirmation Strategy. This involves two phases: *observation* (line 16) and *confirmation* (line 17). Our experiments reveal that when descriptions lack detail for constraint extraction, the LLM may resort to fabricating details, a phenomenon known as hallucination. Drawing inspiration from the Chain-of-thought [40],

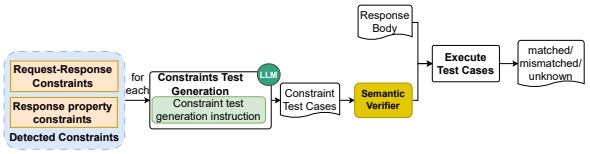

Figure 6: Constraint Test Generation in RBCTest

```
CONSTRAINT_TEST_GEN_PROMPT = '''
Generate a Python script to check if a property in a REST
      API's response meets specified constraints and rules.
Constraint description:
- Constraint from request parameter: {parameter}
- Constraint description: {constraint_description}
API response schema: {response_schema_specification}
The property of the request parameter in API response:
- "{property}": "{prop_description}"
Generate a Python script to verify '{property}' in response.
Rules: {Rules for test gen}
Format the script as shown: ...'''
```

Figure 7: Constraint Test Generation Prompts

we divide the task of extracting constraints into two phases of **observation** and **confirmation**, the initial prompt better contextualizes the description of constraints, enabling the subsequent prompt to more accurately determine a constraint.

In the observation phase (Block 6), the LLM is prompted to identify constraints from the description. For example, given a property `date` (type: `string`) described as *"ISO date: the literal date of the holiday"*, the LLM might infer: *"The date must follow the YYYY-MM-DD format for validity, ensuring consistency within the API."*

Next, the observation is fed into the Constraint Confirmation Prompt (Block 7), where the LLM validates whether the extracted constraint provides enough detail for script generation. This step, similar to Figure 5, ensures that constraints specify values, ranges, or formats. If confirmed, the constraint is marked as a Response Property constraint and stored in the knowledge base.

6 Constraint Test Generation

Taking the mined constraints, RBCTest generates test cases for automated verification of API response bodies against their constraints. This process (Figure 6) uses LLMs to convert Request-Response and Response Property constraints into test cases. Each resulting test case is a validation function that takes response data and request parameters as input, and returns whether the response data satisfies their corresponding mined constraints. We also incorporate a semantic verifier that cross-checks test cases against examples in the OAS file. If a test case fails to validate a given correct example, it suggests that the mined constraint may be incorrect.

Request-Response Constraints Testing. Such cases verify dependencies between a request parameter and its response property. The prompt to generate such test cases (Figure 7) requires four inputs: API input parameter, mined constraints, its property, and response schema. To ensure robustness, RBCTest guides the LLM with predefined rules: using a `try-catch` block for error handling, excluding examples, and following a standardized function template. These rules ensure consistency and focus on constraint verification.

Response Property Constraints Testing. Test cases for Response Property constraints are used to verify response data against

their constraints. To create a test case for this type of constraint, RBCTest uses the prompt depicted in Figure 7. This prompt requires three inputs: the response property, its mined constraints, and response schema. The response property and its mined constraints provide the necessary information for generating validation code, while the response data schema defines the structure and type of the expected data, allowing the LLM to parse response data correctly.

7 Empirical Evaluation

To evaluate RBCTest, we seek to answer the following questions:

RQ1. [Constraints Mining Accuracy]. How well does RBCTest mine the constraints to be used as oracles for verifying API responses compared with the dynamic approach AGORA+ [10]?

RQ2. [Constraints Test Generation Accuracy]. How well does RBCTest generate test cases from the mined constraints?

RQ3. [Accuracy in Test Generation from Correctly Mined Constraints]. How accurate are test cases from correct constraints?

RQ4. [Usefulness in Response Body Testing]. How useful is RBCTest in detecting mismatches between the specification and its working APIs?

RQ5. [Ablation Study]. How much do Observation-Confirmation and Semantic Verifier contribute to RBCTest's performance?

Datasets. We used the following two datasets:

1. AGORA+ dataset [10] has 7 API services with 11 operations.

2. RBCTest dataset [8]. We curate the dataset from eight real-world services from GitLab and Stripe, comprising 65 endpoints and 90 operations from 'Canada Holidays' [28], GitLab-services [18, 23, 28, 29, 41, 42], and 'Stripe' [28, 34, 35]. These services were selected for several reasons: (1) They feature complex request-response structures across diverse business domains. (2) They have been used in API testing. (3) Their active status allows API calls to collect real response data. (4) Their specifications vary in documentation quality, enabling evaluation across different levels of completeness. Stripe's test mode offers limited endpoints and often returns deeply nested, incomplete schemas that miss inputs, hindering validation, so we restricted our study to 8 endpoints without nested schemas.

For both datasets, we manually examine the specifications and build the ground truth of correct constraints, i.e., test oracles. During examination, we systematically used the following criteria to look for and derive the correct and sufficient constraints:

(1) Request-Response Property constraint: This type of constraint on a request parameter must correspond to a property in the response data that reflects this constraint.

(2) Range-of-Value constraint: The type of constraint must specify all possible values or provide a specific data range.

(3) Data Format constraint: This must describe the constraints on data format or refer to widely used formats (e.g., ISO, Unix).

(4) Computed Value constraint: The constraint describes how the value of a field is derived from other fields via computation.

(5) Name-based constraint: This primarily focuses on the constraints held within field names like `id`, `code`, `email`, `url`, `uri`, etc.

We further referred to the oracles available in the AGORA+ dataset to define and report the oracle categories in the RBCTest dataset (Table 1). The first category is the input/output constraints, which describe the logical relations between the input parameters of APIs and the output properties in their response bodies. The remaining categories contain only the constraints/oracles on the output properties in the response bodies. Specifically, the second category contains the constraints on the individual variables in the output. This category contains *unary* constraints belonging to several different data types and templates (*string, boolean, float, time, url, email, etc*). The third category contains the *N-ary (binary or more)* atomic constraints, i.e., involving two or more variables (Table 1). The atomic constraints include comparisons, membership/type checks, and predicate calls without any `AND`, `OR`, or universal/existential quantifiers (negations, if any, are pushed inside as part of the atomic). The last category is the super set of the three preceding categories. A *composite* constraint is expressed as a conjunctive/disjunctive form of the two or more preceding categories, i.e., logical `AND`/`OR` operations are applied among these categories.

Table 1: Categories of Oracles in the RBCTest dataset

Category	Test Oracles and Examples
Input-Output	Constraints involving the relation between input parameters and output properties. E.g.: `input.id_after > return.id`
Single-variable	DefaultValue, isUrl, isDate, isDateTime, String_Specific_Length, Template-Literals, ArrayTypeOracle_{isString, isBoolean, isNumber}, Value-In-Range, Value-In-Set, isBoolean, isNumber, isUnixTime, Array_Specific_Sizes, IsTime, isEmail, etc.
N-ary atomic	Constraints involving two or more variables. E.g.: `sellingTotal = total + margins + markup + totalFees - discounts`
Composite	Constraints with multiple I/O, single-variable, or atomic ones. E.g.: `input.created_after < output.created_at` and `output.created_at` must isDateTime

Table 2: RBCTest's Constraints and AGORA+'s Invariants

ID	Constraints	Invariants
1	the number of returned items has to be less than or equal to the requested limit	input.limit >= size(return.items[])
2	–	return.total >= size(return.items[])
3	return.total is an integer larger than or equal to 1	return.total >= 1
4		return.total is Integer
5	–	input.market is a substring of return.href
6	number of adult guests (1-9) per room	return.adults is Integer
7	–	return.ci_forward_deployment_enabled = true
8	An amount is a positive integer can be up to eight digits	– –
9	–	return.milestone = null
10	–	return.labels[] == []
11	–	return.statistics.wiki_size one of { 0, 41943 }

8 Experimental Results

8.1 Constraint Mining Accuracy (RQ1)

8.1.1 Procedure. For the AGORA/AGORA+ dataset, we ran RBCTest with GPT-4o under the temperature of zero and top_p=0.95 to obtain constraints and manually evaluated them against the ground truth. For the AGORA+ tool, we directly used the results reported in its TOSEM paper [10]. For the RBCTest dataset, we ran our tool in the same setting as with the other dataset. For the AGORA+ tool, we checked its publicly available repository and used RESTest [33] configuration set by AGORA+'s authors to generate 50 request-response pairs following their online instruction. Using this configuration (rather than the default RESTest configuration) ensures the realistic pairs for AGORA+ in RBCTest's dataset. We chose 50 pairs

Table 3: Constraints Mining on the AGORA+ dataset (RQ1)

API	RBCTest						AGORA+				Overlap			Unique	
	GT	TP	FP	FN	P(%)	R(%)	No.I	No.	TP	P(%)	+S	+D	Eq	S	D
G.C	100	94.6	7.4	4.4	92.7	95.6	193	97	97	100	6.8	4.0	49.4	34.4	36.8
G.G	157	149	4.2	7.0	97.3	94.5	145	79	68	86.1	2.8	3.0	43.6	99.6	18.6
A.M	59	43.8	4.4	9.2	90.9	77.9	137	63	31	49.2	8.8	2.6	3.6	28.8	16.0
M.C	50	35.4	4	9.2	89.8	74.1	104	55	31	56.4	6.2	1.0	8.0	20.2	15.8
O.I	15	11.8	0.8	2.4	93.7	78.7	17	14	12	85.7	3.2	1.0	2.2	5.4	5.6
O.S	6	5.2	1.8	0.8	74.3	86.7	6	4	3	75.0	1.0	1.0	1.0	2.2	0.0
S.C	27	19.8	6.4	5.4	75.6	74.4	41	21	21	100	6.6	1.0	2.0	10.2	11.4
S.T	21	17.8	4.0	4.2	81.7	80.2	50	24	19	79.2	2.8	5.8	3.8	5.4	6.6
S.A	16	15.4	3.4	1.0	81.9	93.9	68	24	19	79.2	0.0	3.8	8.2	3.4	7.0
Y.B	4	3.2	1.8	0.0	64.0	80.0	60	14	8	57.1	0.0	0.0	1.0	2.2	7.0
Y.V	161	134.2	8.4	23.0	94.1	84.5	132	78	49	62.8	7.2	0.0	10.4	116.6	31.4
Tot.	**616**	**530.2**	**46.6**	**65.8**	**85.1**	**83.7**	**953**	**473**	**358**	**75.7**	**45.4**	**23.2**	**133.2**	**328.4**	**156.2**

G.C: Github_createOrgRepo, G.G: Github_getOrgRepo, A.M: AmadeusHotel_getMultiHotel, M.C: Marvel_getComicIndividual, O.I: OMDB_byIdOrTitle, O.S: OMDB_bySearch, S.C: Spotify_createPlaylist, S.T: Spotify_getAlbumTracks, S.A: Spotify_getArtistAlbums, Y.B: Yelp_getBusinesses, Y.V: YouTube_listVideos, GT: # constraints in the ground truth, (# of RBCTest constraints grouped by variables=TP+FP). No.I: #r of invariants. No: # of invariants grouped by variables. +S: RBCTest better, +D: AGORA+ better, Eq: Equivalent. S: RBCTest, D: AGORA+.

since its authors reported the best tradeoff between performance and running time at this setting compared to other pair counts.

Table 2 shows samples of constraints identified by RBCTest and invariants generated by AGORA+, illustrating their differences and limited compatibility. To enable meaningful comparison, we adopt a variable-based matching approach. In this context, a variable refers to any request parameter, response property, or intermediate value defined within the API specification. In AGORA+, each invariant is associated with a set of variables, as in Table 2, and a given set of variables may be linked to one or more invariants (e.g. Invariants 3-4). In RBCTest, each constraint refers to one or more variables.

If a constraint and an invariant involve disjoint sets of variables, we treat them as being uniquely identified by their respective approaches. If a RBCTest's constraint and an AGORA+'s invariant refer to the same set of variables, we consider them to be *overlapping*. To evaluate these overlapping pairs, we manually inspect both the constraint and the invariant, and then construct test cases to empirically validate or invalidate them. For example, Constraint 6 in Table 2, we generate test cases with zero and ten guests per room. If both are found to be correct, we compare to see which one is stricter (i.e., better). If one is correct and the other incorrect, we consider the correct one as better. We exclude from our comparison any pairs in which both the constraint and invariant are found to be incorrect. For metrics, we used true positives (TP), false positives (FP), false negatives (FN), and precision $P = \frac{TP}{TP+FP}$, recall $R = \frac{TP}{TP+FN}$. We repeated each experiment five times and reported *the mean results*.

8.1.2 Results on the AGORA+ dataset. As seen in Table 3, RBCTest identified an average of 576.8 constraints out of 616, with an average of 530.2 TPs, resulting in a precision of 85.1% and a recall of 83.7%. AGORA+ detected 953 invariants, and 473 invariants after variable-based grouping. Of these, 358 are TPs, resulting in 75.7% precision.

8.1.3 Results on the RBCTest dataset. We manually reviewed API specifications and noted 986 correct constraints (Table 4). Our tool identified *an average* of 746.8 constraints with a precision of 93.6% and a recall of 70.6%. All standard deviations are below 5%.

8.1.4 Overlapping Analysis between RBCTest and AGORA+ results. For the AGORA+ dataset (Table 3), among overlapping ones, 328.4 constraints were uniquely detected by RBCTest, while 156.2 invariants were uniquely identified by AGORA+. Among the shared ones, *on average*, 45.4 from RBCTest are better while 23.2 from AGORA+ are better, indicating that RBCTest and AGORA+ are complementary and also share detected constraints.

Table 4: Constraints Mining on the RBCTest dataset (RQ1)

API	RBCTest						AGORA+				Overlap			Unique	
	GT	TP	FP	FN	P(%)	R(%)	No.I	No.	TP	P(%)	+S	+D	Eq	S	D
C.H	42	40.8	6.0	1.2	87.2	97.1	25	23	23	100	**3.2**	0.0	7.2	**30.4**	12.6
G.B	61	46.2	3.2	14.8	93.5	75.7	215	123	91	74.0	**7.8**	6.0	17.8	14.6	**59.4**
G.C	86	67.0	3.2	19.4	95.4	77.5	313	178	128	71.9	**11.2**	3.2	25.4	27.2	**88.2**
G.G	119	78.2	5.6	41.0	93.3	65.6	550	318	225	70.8	**17.6**	4.0	38.2	18.4	**165.2**
G.I	268	156.6	3.2	111.8	98.0	58.3	1239	637	451	70.8	**38.2**	10.0	44.2	64.2	**358.6**
G.P	248	177.6	4.6	71.2	97.5	71.4	1228	621	376	60.5	**30.8**	2.0	66.2	78.6	**277.0**
G.R	55	44.0	3.4	11.0	92.8	80.0	220	122	88	72.1	**7.2**	2.0	19.2	15.6	**59.6**
S	107	88.4	18.8	20.8	82.5	81.0	123	86	62	72.1	**27.4**	1.6	18.4	**41.0**	14.6
Tot.	986	698.8	48.0	**291.0**	**93.6**	70.6	3913	2108	1444	**74.0**	**143.4**	28.8	236.6	290.0	**1,035.2**

CH: Canada Holidays, GB: GitLab Branch, GC: GitLab Commit, GG: GitLab Groups, GI: GitLab Issues, GP: GitLab Project, GR: GitLab Repository, S: Stripe.GT: # constraints in the ground truth, (# of RBCTest constraints grouped by variables=TP+FP). No.I: #r of invariants. No: # of invariants grouped by variables. +S: RBCTest better, +D: AGORA+ better, Eq: Equivalent. S: RBCTest, D: AGORA+.

For the RBCTest dataset, regarding the overlapping results in Table 4, *RBCTest and AGORA+ are also complementary*, as RBCTest produces 143.4 better oracles with 290.0 uniquely detected ones, and AGORA+ has 28.8 better oracles with 1,035.2 unique ones.

An example of such complementary nature is the Spotify API. Its documentation states that three predefined image sizes (640, 300, 64) are served, but also notes that "the exact size may vary." Here, AGORA+ detected three explicit values, while RBCTest identified the parameter as an integer, complementing each other's findings.

8.1.5 Qualitative Analysis and Comparison. We examined the cases that RBCTest can detect, yet AGORA+ struggled. *First, the constraints uniquely detected by RBCTest* occurred in scenarios where the API specification explicitly includes descriptions of parameters, operations and response properties. AGORA+ relies on diverse runtime API responses, limiting its effectiveness when those responses lack diversity. For instance, in the `GET /api/v1/provinces` endpoint of the Canada Holidays API, AGORA+ inferred only that `LENGTH(return.id)== 2`, whereas RBCTest mined from the specification that `provinces.id` must be one of the enumerated two-letter Canadian province abbreviations (e.g., `"BC"`, `"ON"`, etc.). Similarly, for the attribute `currency` in `AmadeusHotel_getMultiHotelOffers`, AGORA+ detected that the string length is three characters, however, RBCTest mined from the specification that the string must belong to the list of IATA codes (e.g., USD, JPY). Second, the specification describes a constraint in a more complete ranges of values or formulas than the specific values observed at the runtime. For example, in the Amadeus Hotel API, the `roomQuantity` value is specified as *"an integer between 1 and 9."* RBCTest correctly identified this, whereas AGORA+ provided a general invariant of "Numeric". As another example, in the Amadeus Hotel API, the `sellingTotal` is defined as *"= Total + margins + markup + totalFees - discounts."* While AGORA+ simply detected `sellingTotal` as "Numeric", RBCTest's constraints mined from specifications are more specific and correct. RBCTest also infers better string templates, such as the `RunTime` attribute (e.g., `^\d+\smin$`) in `OMDB_byIdOrTitle`. Finally, RBCTest also mines better the constraints on the rarely exercised attributes, such as `template_repository` in `Github_createOrganizationRepository`.

Table 5: Types of Constraints detected by RBCTest

No.	Category/Test Oracles	Count	No.	Category/Test Oracles	Count
1	I/O	309	9	isBoolean	18
2	isUrl	290	10	isNumber	18
3	isDateTime	160	11	isUnixTime	14
4	Value-In-Set	150	12	Template Literals	11
5	Composite	87	13	ArrayTypeOracle_isString	3
6	isDate	45	14	Array_Specific_Sizes	3
7	String_Specific_Length	35	15	N-ary atomic	3
8	Value-In-Range	29	16	isTime	2

We also examine the cases in which AGORA+ performed well while RBCTest struggled. These cases typically involve invariants that validate URL formats, handle fixed-length tokens, and capture inter-attribute relationships—areas where RBCTest often treats such attributes as plain strings without information. The analysis reveals that RBCTest is limited in these scenarios since it relies primarily on shallow attribute descriptions, typically consisting of the attribute name and a simple type declaration (`{type:string}`). For example, when an attribute such as `return.self` is described only with a type, RBCTest is unable to infer underlying constraints.

In contrast, AGORA+ leverages richer execution context and semantic information to detect the kinds of value-specific constraints. These constraints that AGORA+ detected better belong to three main types. *Type-based constraints* ensure that an attribute conforms to a specific data type, such as verifying that `return.owner.gists_url` or `return.self` is of type `Url`. *Value-based constraints* impose conditions on numeric or string values, for instance, enforcing that `LENGTH(return.runners_token) == 29` or that `return.stats.deletions >= 0` and does not exceed `return.stats.total`. Finally, *inter-attribute constraints* capture relationships among multiple fields, including equality (`return.owner.gists_url == return.organization.gists_url`) and substring relationships (e.g., `return.owner.url` being a substring of both `return.owner.followers_url` and `return.owner.following_url`).

8.1.6 Analysis on Types of Constraints Detected by RBCTest. As shown in Table 5, we summarize the distribution of 1,177 true positive constraints identified across 16 main categories in both the AGORA and RBCTest datasets. Among these, the I/O constraint type, which represents logical and dependency relationships between input and output fields within individual APIs, accounts for 309 instances (26.3%). This indicates that RBCTest is effective at mining commonly described I/O relationships within APIs.

The isUrl category constitutes the second largest group, comprising 290 constraints (24.6%), showing the widespread presence of URL-related attributes in the API specifications. The isDateTime constraints, which specify date/time formats and rules, rank next with 160 constraints (13.6%). The Value-In-Set category, containing 150 constraints (12.7%), reflects the common use of enumerated or bounded value restrictions that encode domain-specific semantics such as status codes, country abbreviations, or numeric thresholds in the APIs. Following these, the Composite category includes 87 constraints (7.4%), showing that RBCTest can capture complex logical conditions composed of multiple atomic constraints.

The remaining categories account for about 15.4% of all detected constraints, covering specialized rules such as date formats, string lengths, numeric ranges, and array checks, which reflect finer-grained or domain-specific properties. This result demonstrates that RBCTest can detect a wide variety of common oracles found in real-world API schemas and response bodies.

Table 6: Constraint Test Generation (RQ2), Test Generation from Correct Constraints (RQ3), and Test Outcomes (RQ4) on the AGORA/AGORA+ Dataset.

API	Constraint Test Gen (RQ2)						Test Gen (RQ3) (Correct Constraints)			Outcome (RQ4)		
	TP	FP	FN	P%	R%	F1%	N	✓	P%	✓	×	?
G.C	78	12	17	86.7	82.1	84.3	79	78	98.7	77	0	1
G.G	127	8	27	94.1	82.5	87.9	127	127	100.0	127	0	0
A.M	23	15	22	60.5	51.1	55.4	29	23	79.3	16	2	5
M.C	21	9	18	70.0	53.8	60.9	26	21	80.8	13	0	8
O.I	12	0	3	100.0	80.0	88.9	12	12	100.0	8	4	0
O.S	5	2	1	71.4	83.3	76.9	6	5	83.3	5	0	0
S.C	17	7	8	70.8	68.0	69.4	19	17	89.5	15	0	2
S.T	19	0	6	100.0	76.0	86.4	19	19	100.0	10	0	9
S.A	15	3	2	83.3	88.2	85.7	16	15	93.8	8	1	6
Y.B	4	1	0	80.0	100.0	88.9	4	4	100.0	3	1	0
Y.V	122	13	31	90.4	79.7	84.7	124	122	98.4	79	1	42
Total	443	70	135	**86.4**	**76.6**	**81.2**	461	443	**96.1**	361	**9**	73

For test outcomes: ✓ (Matched), × (Mismatched), and **?** (Unknown).

8.2 Constraint Test Generation Accuracy (RQ2)

8.2.1 Methodology. Using the best-performing result from Section 8.1, we generated validation scripts for all the mined constraints by following the process in Figure 6. We manually evaluated whether each verification script correctly reflects the intended logic described by its corresponding constraint. The evaluation followed the criteria mentioned during the construction of the ground truth as in Section 7. Verification scripts were executed against real-world API responses. Each constraint was verified by 50 API responses.

8.2.2 Results. For the AGORA/AGORA+ dataset, as seen in Table 6, RBCTest performs well in generating test scripts from mined constraints, achieving a validity rate of 86.4% and covering 76.6% of the ground-truth constraints. However, it missed 135 out of 578 constraints. Notably, for some services, e.g., Github_createOrgRepo and Github_getOrgRepo, RBCTest achieved up to 82% recall, while for Yelp, it reached 100% recall.

For the RBCTest dataset, Table 7 shows a precision of 91.7%, indicating that most generated test scripts correctly validate their associated constraints. The recall reached 71.3%, though 279 out of 973 constraints were missed. These metrics are slightly lower than those in RQ1, as this evaluation jointly considers both constraint correctness from the mining and the correctness of test execution.

RBCTest excels in test generation for *Response Property constraints*, which often involve clear format or range-based rules, e.g., "three-letter currency code" or "Unix epoch timestamp." These constraints are easier to detect due to their descriptive nature. In contrast, *Request–Response*, *Range-of-Value*, and *Computed Value* constraints are more error-prone, as they require accurate mappings between request parameters and response properties. However, RBCTest still achieves very high precision on these constraints.

Test generation errors often stem from missing or vague descriptions, especially in services like GitLab, where response property documentation is sparse. For example, the request parameter `avatar` (used for image uploads) was incorrectly mapped to the response field `avatar_url` due to name similarity, despite different semantics. Services that define complex *Data Format* constraints—such as

Table 7: Constraint Test Generation (RQ2), Test Generation from Correct Constraints (RQ3), Test Outcomes (RQ4) on the RBCTest Dataset.

API	Constraint Test Gen (RQ2)						Test Gen (RQ3) (Correct Constraints)			Outcome (RQ4)		
	TP	FP	FN	P%	R%	F1%	N	✓	P%	✓	×	?
C.H	36	0	6	100	85.7	92.3	36	36	100.0	22	2	12
G.B	47	6	13	88.2	75.0	84.7	47	45	95.7	42	1	2
G.C	64	7	16	90.1	75.3	88.5	69	64	92.8	51	6	7
G.G	89	6	26	93.7	76.1	85.8	91	89	97.8	78	6	4
G.I	171	8	91	95.5	64.3	78.7	175	171	97.7	132	19	20
G.P	163	16	70	91.1	68.5	80.6	168	163	97.0	139	0	24
G.R	44	7	11	86.3	80.0	83.0	46	44	95.7	41	0	2
S	82	13	26	86.3	74.5	82.0	84	82	97.6	68	3	11
Total	694	63	259	**91.7**	**71.3**	**82.5**	716	694	**96.9**	573	37	82

For test outcomes: ✓ (Matched), × (Mismatched), and **?** (Unknown).

"`numberOfNights` must always be greater than 0," or fields expected to be unique like `id`, `code` or `barCode`—also present challenges when descriptions are missing or implicit. Attributes with special rules (*Name-based constraint*) like `latitude` or `longitude` in `AmadeusHotel_get-MultiHotel` or `lang` in Marvel were either overlooked or led to test scripts that failed to express the intended constraint logic. In these cases, the lack of structured or informative descriptions in the specification hindered constraint detection and test generation.

8.3 Test Generation Accuracy from Correctly Mined Constraints (RQ3)

8.3.1 Methodology. The experiment in RQ2 evaluates the entire process from constraint mining to test generation. This RQ3 focuses only on evaluating the test generation for the constraints *correctly mined by RBCTest*. We perform test generation from those correct constraints. The generated test cases are manually evaluated if they correctly verify the associated constraints. We used the following rules to check if a test case is correct:

(1) Test Input: The generated test case is correct if it correctly receives two inputs for Request-Response constraints: 1) requested information and 2) response data, and one input for Response Property constraints (response data).

(2) Constraint Handling: The generated test case must cover all conditions in the constraint.

(3) Test Output: Each script must be evaluated on 50 example responses collected from real-world data. The test must return:

i) 0 (`unknown`) if lacking of sufficient data for condition checking (e.g., empty or `null` value).

ii) 1 (`matched`) if the provided input satisfies the constraint.

iii) -1 (`mismatched`) if the provided input does not satisfy it.

The final result is considered `matched` only if all test cases return 1 and none return -1; any case with 0 is acceptable. If all cases return 0, the result is `unknown`. If any case returns -1, the result is considered as `mismatched`. We only consider the set of test cases derived from valid constraints as identified in RQ2 (denoted as N in Tables 6–7).

8.3.2 Results. As seen in Table 7, RBCTest generates 714 TP test cases out of 746 in the RBCTest dataset, resulting a precision of 94.3%. In 4/8 services, it achieves a precision of 96%, while the lowest precision is in `GitLab Repository API`, with a precision of 83%. For the AGORA/AGORA+ dataset, it generates 443 correct test cases out of 513, with a precision of 96.1%. In two services, it achieves 100%.

Note that in RQ2, we consider all mined constraints, while in RQ3, we use only the correctly mined ones. Thus, the results in RQ3 (e.g., precision of 96.1% on the AGORA dataset) are higher than those in RQ2 (e.g., precision of 86.4%).

Our analysis reveals that RBCTest excels in generating code for *Response Property constraints*, which mainly involve format or value range validation. In contrast, *Request-Response constraints* are more error-prone due to the complex logic required to parse and verify dependencies between request parameters and response properties. Further analysis reveals that test generation errors primarily stem from (1) missing descriptions and (2) ambiguous keywords.

Consider the `'get-/issues'` endpoint in GitLab Issues, where a constraint exists between the request parameter `'due_date'` and a response property of the same name. The parameter is described as *"Accepts: 0 (no due date), overdue, week, month, next_month,"* while the response property lacks a description. As a result, our tool generates test cases to validate the property against this constraint, even though `'due_date'` in the response is a date-time string, leading to incorrect tests. Such errors are common in GitLab due to insufficient descriptions for response properties.

In the same `'get-/issues'` endpoint, the `'milestone'` parameter is described as *"The milestone title. None lists all issues with no milestone. Any lists all issues that have an assigned milestone."* The API filters returned issues based on `'None'` or `'Any'` from the request parameter. However, RBCTest misinterpreted `'None'` as Python's `null`, leading to errors. This mistake in `'milestone'` propagated to 13 other incorrect test cases due to its involvement in multiple operations.

8.4 Usefulness: Detecting Mismatches between Constraints and API Responses (RQ4)

8.4.1 Methodology. In this experiment, we use RBCTest's generated test cases to detect mismatches between the API specification and API response bodies. A mismatch often indicates either that 1) the specification is outdated; or 2) the APIs' implementation does not align with the constraints in their specification. To this goal, we performed the following. For the RBCTest dataset, for each API endpoint, we used RESTest [33] to generate 50 API requests, executed the SUT with APIs, and obtained the actual responses. For each execution, we collected 1) request data, and 2) actual response data. We used only the pairs of request-response with the status code of 2xx. Response data contains multiple properties, each attached with constraints and their test cases generated by RBCTest. We ran the test cases (in RQ3) *generated for the constraints on the properties in the response data to collect the outcomes*. If the result is false, we report a mismatch. Otherwise, it is a match.

For the AGORA/AGORA+ dataset, we used the request-response pairs provided on its repository, whose responses were obtained by its authors from the actual execution. We ran the test cases generated by RBCTest and compared with the responses as described.

8.4.2 Results. For the RBCTest dataset, we performed test runs on all 694 correctly generated test cases from RQ3 (Table 7). The tests evaluate if the response data conforms to the constraints detected from the API specification. Our test results indicate 573 'matched' response bodies (i.e., consistent with the specification), 37 'mismatched', and 82 'unknown'. Out of 714 correctly mined constraints, 573 were verified by the test cases, meaning that 82.5%

of the constraints are met by the actual execution of the SUTs. Our tool detected *37 mismatches*, revealing inconsistencies between specifications and execution of the APIs. An 'unknown' occurs when a property is absent in the response body due to its optional nature.

For the AGORA/AGORA+ dataset, we performed test runs on all 443 correctly generated tests from RQ3 (Table 6). Our test results indicate 361 'matched' response bodies (i.e., consistent with the specification), 9 'mismatched', and 73 'unknown'. Out of 443 correctly mined constraints, 361 were verified by the test cases, meaning that 81.5% of the constraints are satisfied by the actual execution of the APIs. Our tool detected *9 mismatches*, revealing inconsistencies between specifications and API executions.

8.4.3 Analysis on Detected Mismatches. For 46 mismatches detected by RBCTest [9], we conducted a detailed analysis by classifying them into 6 constraint categories, as illustrated in Figure 8. The isDateTime category, which refers to the constraint in date and time data types, accounts for the largest proportion (45.7%), indicating that temporal inconsistencies—such as variations in date–time formats or missing temporal fields—are the predominant mismatch between API specifications and their actual responses.

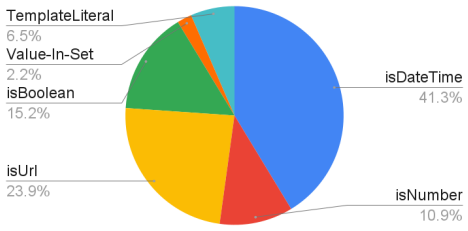

Figure 8: Detected Mismatches by Category

Other categories, including IsUrl (the property as a URL) and IsNumber (each 13.0%), IsBoolean and boolean values (10.9%), and Template Literals (e.g., String templates/lengths: Canadian provincial abbreviations in Canadian_Holidays API), and Value-In-Set (8.7%, e.g., in YouTube, video length in ISO_8601 duration formats: PT#M#S), show lower but still notable occurrences.

These findings suggest that the detected mismatches are non-trivial, and they mainly arise in the fields governed by strict formatting or parsing rules. In particular, the prevalence of date–time errors highlights the need for better normalization and schema-level validation. Beyond format-related issues, several mismatches may also stem from incorrect implementation or schema drift, where API implementations change faster than their specifications. This result indicates that *RBCTest is useful in continuous validation and automated schema synchronization and in maintaining consistency between API behavior and its declared constraints.*

8.4.4 Root-cause Analysis. We reported the root causes of these mismatches: (1) incompatible data formats, (2) implicitly-described nullable properties, and (3) inter-parameter request dependencies.

(1) Incompatible Data Formats: Our tool detected constraint mismatches mainly from GitLab services. For instance, the constraint *"date will be returned in ISO 8601 format* `YYYY-MM-DDTHH:MM"` appears in GitLab operations. However, the actual data format is `"2012-09-20T08:50:22.000Z"`, which includes a decimal part for seconds, leading to an inconsistency. Interestingly, we found that three instances of this type of inconsistency **were reported as the issues** on the GitLab forum [1, 3, 4]. This is *anecdotal evidence on RBCTest's usefulness in detecting real-world data-format issues.*

(2) Implicitly-Described, Nullable Properties: This issue is common in GitLab, where some response properties are nullable but lack descriptions in the specification, leading to parsing errors. Notably, these issues have been reported on the GitLab forum [2].

(3) Inter-Parameter Request Dependencies: This was found in only one case. For the operation `'GET/groups'` from GitLab Group, there is a constraint on the request parameter `"order_by"` affecting the `"name"` property in the response data. This logic dictates that the array of groups in the API response should be sorted according to the `"order_by"` parameter. RBCTest checked only one-to-one conditions among parameters, whereas the sorting order depends on both `"order_by"` and `"sort"` parameters (specifying the sort direction). As a result, this resulted in a detected mismatch. Future work could address more complex dependencies among multiple parameters.

Table 8: Ablation Study on the RBCTest dataset (RQ5)

Variant	Constraint Test Generation					
	TP	FP	FN	P	R	F1
RBCTest-	846	279	706	75.2%	54.5%	63.2%
RBCTest	1,137	133	394	89.5%	74.3%	81.2%

8.5 Ablation Study (RQ5)

In this experiment, we aim to compare two variants: 1) (RBCTest$^-$): RBCTest without observation-confirmation (OC) prompting and semantic verifier, and 2) RBCTest with both components. To build the former, we modify the prompt for constraint mining in Figure 4 as follows. Instead of providing GPT-4o with observations derived from another prompt, we feed the description of the parameters and response schema as in the API specification. This prompt replaces Blocks ❶-❺ from Figure 3. After providing data on a property, we instruct the LLM to decide if the given property contains a constraint and if there is enough to verify it. This modification merges steps ❻ and ❼ into a single step. The output is *yes* or *no*. We ran both variants five times and recorded the mean results in the same process as described in the previous sections.

As seen in Table 8, the elimination of observation-confirmation prompting and semantic verifier affects the outcomes, as evidenced by a reduction of 312 correct constraints. Concurrently, there is an increase in the number of false positives. The F1-score and recall of RBCTest- decreased by approximately 25% compared to the original RBCTest. False positives frequently arise when the model mis-maps request parameters to response properties based solely on their names. In addition, there are cases where certain fields have names that misleadingly suggest they represent constraints, but based on their descriptions and the predefined evaluation rules, they are actually not constraints. Such mistakes are less common in RBCTest, where the observation prompt is used to enhance the property before it is processed by the confirmation prompt. *This result confirms our key contribution of our LLM prompting strategy.*

The semantic verifier's goal is to remove the invalid constraints to improve precision. We used a simple verifying mechanism via examples in API specification (Section 4). We removed the semantic verifier and ran the static component. In the RBCTest dataset with 89 examples, one example invalidated one detected constraint. In the AGORA dataset, with 223 examples over 11 API operations, 6 examples invalidated 6 false-positive constraints. Overall, the verifier accurately confirmed all valid constraints and successfully eliminated 7 incorrect constraints, thus achieving higher precision (86.4% increasing to 87.5%) while maintaining recall 76.6%. These results show that more examples in the API are useful to invalidate more incorrect constraints and confirm the correct ones. In general, other types of semantic verifier can be integrated into our framework such as constraint solvers, SMT solvers, domain-specific checkers (valid zip code, phone number format, or valid date checkers), etc.

8.6 Token Cost Analysis

For **constraint inference**, RBCTest does not rely on LLMs to analyze API responses; it instead operates directly on the API specification. This allows each API response field to be processed only once to generate test oracles, which can then be reused across all subsequent API calls. We measured the marginal inference cost of RBCTest with GPT-4o. Due to that strategy, for 19 APIs from both datasets, RBCTest consumed 1,300,479 input tokens (68,445 per API) and 180,527 output tokens (9,501 per API). The total cost for both datasets was $5.1. [1]

The **test generation** phase is more resource-intensive, consuming a total of 1,332,494 input tokens and 473,521 output tokens, with the former accounting for around 73% of the total. The total LLM cost was estimated at $8.1. The highest token consumption occurred in the GitLab APIs and YouTube_listVideos, with the GitLab Issues API alone using 199,361 input tokens and 70,981 output tokens. Smaller APIs such as Yelp_getBusinesses and OMDB required fewer numbers of tokens.

Overall, the analysis of token usage and monetary cost shows RBCTest's efficiency, with the cost of $13.2 in two phases of constraint inference and test script generation for all APIs in the datasets.

9 Threats to Validity

1. Internal Validity. LLM hallucinations might lead to unexpected outcomes [37]. To mitigate this, we incorporated (1) temperature of zero for GPT-4o, (2) a semantic verifier (Figure 6) and (3) observation-confirmation prompting (Figure 3), which enhance validation through external API specifications and internal consistency checks.

Data leakage is expected to be minimal: while specifications and APIs may appear in pre-training data, the specific pairs of specifications and constraints are not. To address non-determinism from input variation, we consistently used the same prompt templates.

2. External Validity. Our dataset may not fully represent the API landscape, affecting generalizability. Outcomes could vary with different datasets, particularly those with unique constraints. Generated test cases may miss general constraints or edge cases beyond what is explicitly documented. Implicit constraints not in API specifications might also be valid but unaccounted for. External tools may introduce inaccuracies, potentially affecting the results.

[1] $2.5 per 1M input tokens ($1.25 if cached) $10 per 1M output tokens.

10 Discussion

Our goal and empirical results indicate that the static approaches such as RBCTest and the dynamic approaches are *complementary to each other*. First, they complement each other in *different usage scenarios*. The API specifications would be available when 1) API developers provide the OAS specification files for API usages, 2) a tester performs testing based on OAS files, or 3) the front-end developers need to know the specification for back-end integration. If an OAS file is not available, a dynamic approach could extract the constraints from runtime information, e.g., in the regression testing or other scenarios where the application could be correctly executed. In contrast, if the application is under development and non-executable, RBCTest could rely on the OAS files.

Second, they are also *complementary by exploring different information channels*. RBCTest detects constraints from API specifications while a dynamic approach detects invariants through API execution. They provide coverage where the other falls short, either when specifications are incomplete or when execution history is overly specific or unreliable (e.g., during development).

Third, as demonstrated in our *results*, each approach recovers shared constraints as well as unique ones: RBCTest covers cases missed by runtime test cases or when running is infeasible, while AGORA/AGORA+ handles cases absent from the specification.

Finally, *for the mismatch/bug detection application*, they are also complementary. RBCTest detects bugs/mismatches between API specification and implementation. A dynamic one detects bugs across unseen responses in different executions. Thus, we call for a combination between RBCTest and dynamic approaches [22].

11 Conclusion

Novelty. This work, RBCTest, demonstrates the effectiveness of leveraging LLMs to statically mine logical constraints to be used as oracles for API response bodies directly from API specifications, addressing key limitations of dynamic analysis-based approaches. By introducing the Observation-Confirmation prompting scheme, we reduce hallucinations and significantly improve the precision of mined oracles. RBCTest applies LLMs in combination with API documentation to derive *the oracles from the response bodies* without relying on runtime execution, enabling broader and more accurate constraint discovery. This novel use of LLMs for static oracle mining represents a shift from traditional behavior-driven testing toward specification-driven validation. RBCTest uncovers *46 real-world inconsistencies* between documented specifications and actual APIs.

Practical Impacts. Our results also encourage further exploration into **combining static and dynamic techniques**, leveraging the strengths of each, to build hybrid systems that can generate richer test oracles, increase coverage, and surface deeper specification-implementation mismatches across interfaces.

12 Data Availability

Our benchmark and code are publicly available at our website [7].

Acknowledgments

This work is funded by the University of Science, VNU-HCM, under Grant No. CNTT 2025-21. Tien N. Nguyen was supported in part by the US National Science Foundation (NSF) grant CNS-2120386.

References

[1] 2024. GitLab Issue 19667. https://gitlab.com/gitlab-org/gitlab/-/issues/19667
[2] 2024. GitLab Issue 348376. https://gitlab.com/gitlab-org/gitlab/-/issues/348376
[3] 2024. GitLab Issue 349664. https://gitlab.com/gitlab-org/gitlab/-/issues/349664
[4] 2024. GitLab Issue 410638. https://gitlab.com/gitlab-org/gitlab/-/issues/410638
[5] 2024. Katalon Studio Automation Testing Tool. https://katalon.com/
[6] 2024. Postman API Testing Tool. https://www.postman.com/
[7] 2025. RBCTest. https://github.com/vnkata/RBCTest
[8] 2025. RBCTest. https://github.com/vnkata/RBCTest/blob/main/approaches/OraclesDis.png
[9] 2025. RBCTest. https://github.com/vnkata/RBCTest/blob/main/approaches/list_bug.xlsx
[10] Juan C. Alonso, Michael D. Ernst, Sergio Segura, and Antonio Ruiz-Cortés. 2025. Test Oracle Generation for REST APIs. *ACM Trans. Softw. Eng. Methodol.* 35, 1, Article 19 (Dec. 2025), 37 pages. doi:10.1145/3726524
[11] Juan C. Alonso, Alberto Martin-Lopez, Sergio Segura, Gabriele Bavota, and Antonio Ruiz-Cortés. 2025. SATORI: Static Test Oracle Generation for REST APIs. In *Proceedings of the 2025 Automated Software Engineering Conference (ASE'25)*. ACM Press.
[12] Juan C Alonso, Alberto Martin-Lopez, Sergio Segura, Jose Maria Garcia, and Antonio Ruiz-Cortes. 2022. ARTE: Automated Generation of Realistic Test Inputs for Web APIs. *IEEE Transactions on Software Engineering* 49, 1 (2022), 348–363.
[13] Juan C. Alonso, Sergio Segura, and Antonio Ruiz-Cortés. 2023. AGORA: Automated Generation of Test Oracles for REST APIs. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Seattle, WA, USA) *(ISSTA 2023)*. Association for Computing Machinery, New York, NY, USA, 1018–1030. doi:10.1145/3597926.3598114
[14] Andrea Arcuri. 2019. RESTful API automated test case generation with EvoMaster. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 28, 1 (2019), 1–37.
[15] Andrea Arcuri. 2020. Automated black-and white-box testing of restful apis with evomaster. *IEEE Software* 38, 3 (2020), 72–78.
[16] Andrea Arcuri and Juan P Galeotti. 2020. Handling SQL databases in automated system test generation. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 29, 4 (2020), 1–31.
[17] Andrea Arcuri and Juan P Galeotti. 2021. Enhancing search-based testing with testability transformations for existing APIs. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 31, 1 (2021), 1–34.
[18] Vaggelis Atlidakis, Patrice Godefroid, and Marina Polishchuk. 2019. RESTler: stateful REST API fuzzing. In *Proceedings of the 41st International Conference on Software Engineering* (Montreal, Quebec, Canada) *(ICSE '19)*. IEEE Press, 748–758. doi:10.1109/ICSE.2019.00083
[19] Adeel Ehsan, Mohammed Ahmad ME Abuhaliqa, Cagatay Catal, and Deepti Mishra. 2022. RESTful API testing methodologies: Rationale, challenges, and solution directions. *Applied Sciences* 12, 9 (2022), 4369.
[20] Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. 2007. The Daikon system for dynamic detection of likely invariants. *Sci. Comput. Program.* 69, 1–3 (dec 2007), 35–45. doi:10.1016/j.scico.2007.01.015
[21] Amid Golmohammadi, Man Zhang, and Andrea Arcuri. 2023. Testing RESTful APIs: A Survey. *ACM Trans. Softw. Eng. Methodol.* 33, 1, Article 27 (Nov. 2023), 41 pages. doi:10.1145/3617175
[22] Hieu Huynh, Tri Le, Tu Nguyen, Viet Nguyen, Vu Nguyen, and Tien N. Nguyen. 2025. RBCTest: Leveraging LLMs to Mine and Verify Oracles of API Response Bodies for RESTful API Testing. arXiv:2504.17287 [cs.SE] https://arxiv.org/abs/2504.17287
[23] Stefan Karlsson, Adnan Causevic, and Daniel Sundmark. 2020. QuickREST: Property-based Test Generation of OpenAPI-Described RESTful APIs . In *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*. IEEE Computer Society, Los Alamitos, CA, USA, 131–141. doi:10.1109/ICST46399.2020.00023
[24] Myeongsoo Kim, Davide Corradini, Saurabh Sinha, Alessandro Orso, Michele Pasqua, Rachel Tzoref-Brill, and Mariano Ceccato. 2023. Enhancing REST API Testing with NLP Techniques. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Seattle, WA, USA) *(ISSTA 2023)*. Association for Computing Machinery, New York, NY, USA, 1232–1243. doi:10.1145/3597926.3598131
[25] Myeongsoo Kim, Saurabh Sinha, and Alessandro Orso. 2025. LlamaRestTest: Effective REST API Testing with Small Language Models. *Proc. ACM Softw. Eng.* 2, FSE, Article FSE022 (June 2025), 24 pages. doi:10.1145/3715737
[26] Myeongsoo Kim, Tyler Stennett, Dhruv Shah, Saurabh Sinha, and Alessandro Orso. 2024. Leveraging Large Language Models to Improve REST API Testing. In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results* (Lisbon, Portugal) *(ICSE-NIER'24)*. Association for Computing Machinery, New York, NY, USA, 37–41. doi:10.1145/3639476.3639769
[27] Myeongsoo Kim, Qi Xin, Saurabh Sinha, and Alessandro Orso. 2022. Automated test generation for REST APIs: no time to rest yet. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis* (Virtual, South Korea) *(ISSTA 2022)*. Association for Computing Machinery, New York, NY, USA, 289–301. doi:10.1145/3533767.3534401
[28] Tri Le, Thien Tran, Duy Cao, Vy Le, Tien N. Nguyen, and Vu Nguyen. 2024. KAT: Dependency-Aware Automated API Testing with Large Language Models . In *2024 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE Computer Society, Los Alamitos, CA, USA, 82–92. doi:10.1109/ICST60714.2024.00017
[29] Jiaxian Lin, Tianyu Li, Yang Chen, Guangsheng Wei, Jiadong Lin, Sen Zhang, and Hui Xu. 2022. foREST: A Tree-based Approach for Fuzzing RESTful APIs. *arXiv preprint arXiv:2203.02906* (2022).
[30] Yi Liu, Yuekang Li, Gelei Deng, Yang Liu, Ruiyuan Wan, Runchao Wu, Dandan Ji, Shiheng Xu, and Minli Bao. 2022. Morest: model-based RESTful API testing with execution feedback. In *Proceedings of the 44th International Conference on Software Engineering* (Pittsburgh, Pennsylvania) *(ICSE '22)*. Association for Computing Machinery, New York, NY, USA, 1406–1417. doi:10.1145/3510003.3510133
[31] Bogdan Marculescu, Man Zhang, and Andrea Arcuri. 2022. On the Faults Found in REST APIs by Automated Test Generation. *ACM Trans. Softw. Eng. Methodol.* 31, 3, Article 41 (March 2022), 43 pages. doi:10.1145/3491038
[32] Alberto Martin-Lopez, Andrea Arcuri, Sergio Segura, and Antonio Ruiz-Cortes. 2021. Black-Box and White-Box Test Case Generation for RESTful APIs: Enemies or Allies? . In *2021 IEEE 32nd International Symposium on Software Reliability Engineering (ISSRE)*. IEEE Computer Society, Los Alamitos, CA, USA, 231–241. doi:10.1109/ISSRE52982.2021.00034
[33] Alberto Martin-Lopez, Sergio Segura, and Antonio Ruiz-Cortés. 2021. RESTest: automated black-box testing of RESTful web APIs. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis* (Virtual, Denmark) *(ISSTA 2021)*. Association for Computing Machinery, New York, NY, USA, 682–685. doi:10.1145/3460319.3469082
[34] Alberto Martin-Lopez, Sergio Segura, and Antonio Ruiz-Cortés. 2022. Online testing of RESTful APIs: promises and challenges. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Singapore, Singapore) *(ESEC/FSE 2022)*. Association for Computing Machinery, New York, NY, USA, 408–420. doi:10.1145/3540250.3549144
[35] A. Giuliano Mirabella, Alberto Martin-Lopez, Sergio Segura, Luis Valencia-Cabrera, and Antonio Ruiz-Cortés. 2021. Deep Learning-Based Prediction of Test Input Validity for RESTful APIs. In *2021 IEEE/ACM Third International Workshop on Deep Learning for Testing and Testing for Deep Learning (DeepTest)*. 9–16. doi:10.1109/DeepTest52559.2021.00008
[36] Omur Sahin and Bahriye Akay. 2021. A Discrete Dynamic Artificial Bee Colony with Hyper-Scout for RESTful web service API test suite generation. *Appl. Soft Comput.* 104, C (June 2021), 17 pages. doi:10.1016/j.asoc.2021.107246
[37] June Sallou, Thomas Durieux, and Annibale Panichella. 2024. Breaking the Silence: the Threats of Using LLMs in Software Engineering. In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results* (Lisbon, Portugal) *(ICSE-NIER'24)*. Association for Computing Machinery, New York, NY, USA, 102–106. doi:10.1145/3639476.3639764
[38] Abhinav Sharma, M Revathi, et al. 2018. Automated API testing. In *2018 3rd International Conference on Inventive Computation Technologies (ICICT)*. IEEE, 788–791.
[39] Emanuele Viglianisi, Michael Dallago, and Mariano Ceccato. 2020. RESTTESTGEN: Automated Black-Box Testing of RESTful APIs . In *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*. IEEE Computer Society, Los Alamitos, CA, USA, 142–152. doi:10.1109/ICST46399.2020.00024
[40] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed H. Chi, Quoc Le, and Denny Zhou. 2022. Chain of Thought Prompting Elicits Reasoning in Large Language Models. *CoRR* abs/2201.11903 (2022). arXiv:2201.11903 https://arxiv.org/abs/2201.11903
[41] Huayao Wu, Lixin Xu, Xintao Niu, and Changhai Nie. 2022. Combinatorial testing of RESTful APIs. In *Proceedings of the 44th International Conference on Software Engineering* (Pittsburgh, Pennsylvania) *(ICSE '22)*. Association for Computing Machinery, New York, NY, USA, 426–437. doi:10.1145/3510003.3510151
[42] Koji Yamamoto. 2021. Efficient penetration of API sequences to test stateful RESTful services. In *2021 IEEE International Conference on Web Services (ICWS)*. 734–740. doi:10.1109/ICWS53863.2021.00101
[43] Man Zhang and Andrea Arcuri. 2021. Adaptive Hypermutation for Search-Based System Test Generation: A Study on REST APIs with EvoMaster. *ACM Trans. Softw. Eng. Methodol.* 31, 1, Article 2 (Sept. 2021), 52 pages. doi:10.1145/3464940
[44] Man Zhang and Andrea Arcuri. 2021. Enhancing Resource-Based Test Case Generation for RESTful APIs with SQL Handling. In *Search-Based Software Engineering: 13th International Symposium, SSBSE 2021, Bari, Italy, October 11–12, 2021, Proceedings* (Bari, Italy). Springer-Verlag, Berlin, Heidelberg, 103–117. doi:10.1007/978-3-030-88106-1_8

[45] Man Zhang, Bogdan Marculescu, and Andrea Arcuri. 2019. Resource-based test case generation for RESTful web services. In *Proceedings of the Genetic and Evolutionary Computation Conference* (Prague, Czech Republic) *(GECCO '19)*. Association for Computing Machinery, New York, NY, USA, 1426–1434. doi:10.1145/3321707.3321815

[46] Man Zhang, Bogdan Marculescu, and Andrea Arcuri. 2021. Resource and dependency based test case generation for RESTful Web services. *Empirical Softw. Engg.* 26, 4 (July 2021), 61 pages. doi:10.1007/s10664-020-09937-1